

第十章 数据库恢复技术

(2026 春季)



- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

10.1 事务的基本概念

- 事务

- 事务的**ACID**特性

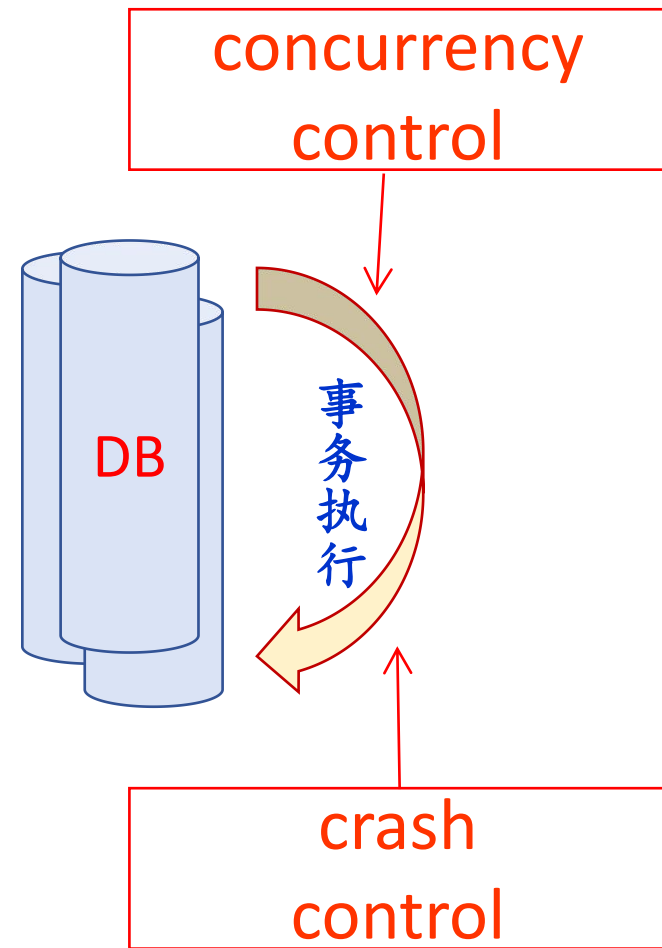
事务 (1)

- ❑ **事务(Transaction)**是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。
- ❑ **事务和程序**是两个概念
 - 在关系数据库中，一个事务可以是一条**SQL**语句，一组**SQL**语句或整个程序
 - 一个程序通常包含多个事务
- ❑ **事务**是恢复和并发控制的基本单位

事务 (2)

□ 为什么要引入‘事务’概念？

- 为了确保实现数据库的数据完整性 (integrity of the database)
- 在一个数据库系统中，如果所有数据都满足数据完整性约束的定义，那么就称数据库处于一个一致性状态(consistent state)
- 影响数据完整性的三个因素
 - concurrency
 - abort
 - crash
- 在数据库管理系统中引入事务处理功能，可以确保在任何情况下，都能保证数据库中数据的正确性。



事务的定义

□ 在现有的数据库管理系统中，事务的定义方式如下：

➤ 事务的启动：隐式方式

- 由数据库管理系统来决定，何时为何用户启动一个新的事务

➤ 事务的结束

- 用户调用commit或rollback命令，显式地结束当前事务
- 由数据库管理系统强行结束一个用户的当前正在运行中的事务

事务的启动

❑ 事务的启动方式是隐式的，一旦出现以下三种情况之一，系统将创建一个新的事务来执行用户的访问请求。

① 数据定义命令(DDL)

- 每一条数据定义命令都将被作为一个单独的事务来执行
- 在此之前，用户当前正在运行的事务(active事务)将被自动提交

② 将系统设为自动提交方式（打开自动提交标志）

- 每一条数据库访问命令都将被作为一个单独的事务来执行，并根据执行结果由DBMS自动完成提交或回退

③ 数据操纵命令(DML)

- 当DBMS接收到来自某个用户的数据操纵请求时，首先检查该用户当前有没有正在运行的active事务：
- 如果有，该请求被作为当前active事务的组成部分去执行；
- 如果没有，DBMS将自动为该用户启动一个新的事务，该请求就被作为新事务的第一条数据访问请求去执行。

事务的结束 (1)

❑ COMMIT

- 事务正常结束
- 提交事务的所有操作（读+更新）
- 被提交的事务对数据库的所有更新结果被写回到磁盘上的物理数据库中，确保事务执行结果的持久化

❑ ROLLBACK

- 事务异常终止
- 事务运行的过程中发生了故障，不能继续执行
- 数据库管理系统将负责撤销被异常终止的事务对数据库的所有已完成的操作，并将数据库中的数据回退到事务开始时状态

事务的结束 (2) 提交

□ Commit transaction

- 用于提交用户当前的active事务。如果能够成功完成提交，该事务在执行过程中对于数据库的所有修改操作结果都将永久地反映到数据库中，并且不可被取消。
- 事务的提交操作也可能失败，其原因包括：
 - 发生系统故障
 - 在提交阶段执行的数据完整性检查不满足要求
- 在事务提交失败后，用户可以通过回退(Rollback)操作来取消当前事务
 - 由系统自动提交的事务，如果提交失败，系统将自动执行事务的回退操作

事务的结束 (3) 回滚

□ Rollback transaction

- 取消当前**active**事务在执行过程中的所有操作结果，将数据库回滚至该事务开始前的状态，以便重新执行或放弃（**abort**）该事务
- 事务的执行过程不可能被‘回滚’，执行**rollback**操作是为了撤销该事务在之前的执行过程中对数据库中的数据所作的修改

□ 保存点 (savepoint)

- 在事务的执行过程中，用户可以为该事务设置若干个保存点
- 用户事务可以使用 **Rollback** 命令将当前事务回退到前面的某个保存点**sp**，放弃“在保存点**sp**之后，回退操作之前”执行过的对数据库的所有访问操作，并继续执行当前事务
- 不带保存点的回退操作将结束并放弃整个事务

事务控制命令

❑ 除了上述三条事务控制语句外，数据库管理系统通常还会提供下述几条与事务有关的控制命令(DCL)

① 设置事务的自动提交命令

SET AUTOCOMMIT ON | OFF ;

② 设置事务的类型

SET TRANSACTION READONLY | READWRITE ;

③ 设置事务的隔离级别

**SET TRANSACTION ISOLATION LEVEL
READUNCOMMITTED |
READCOMMITTED |
READREPEATABLE |
SERIALIZABLE ;**

❑ 用户可以使用这三条命令来设置自己未来的新事务的运行方式！

事务的特性 (ACID特性)

□ 事务的ACID特性:

➤ 原子性 (Atomicity)

- 事务是数据库的逻辑工作单位

- 事务中包括的诸操作要么都做，要么都不做

➤ 一致性 (Consistency)

➤ 隔离性 (Isolation)

➤ 持续性 (Durability)

一致性

- ❑ 事务执行的结果必须是使数据库从一个一致状态变到另一个一致状态
- ❑ 一致性状态
 - 数据库中只包含成功事务提交的结果
- ❑ 不一致状态
 - 数据库系统运行中发生故障，有些事务尚未完成就被迫中断；
 - 这些未完成事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态

一致性 与 原子性

❑ 银行转帐：从帐号A中取出一万元，存入帐号B。

➤ 定义一个事务，该事务包括两个操作

A	B
$A = A - 10000$	
	$B = B + 10000$

➤ 这两个操作要么全做，要么全不做

- 全做或者全不做，数据库都处于一致性状态。
- 如果只做一个操作，用户逻辑上就会发生错误，少了一万元，数据库就处于不一致性状态。

□ 隔离性

- 一个事务的执行不能被其他事务干扰
- 一个事务内部的操作及使用的数据对其他并发事务是隔离的
- 并发执行的各个事务之间不能互相干扰

□ 持续性 也称 永久性、持久性 (Permanence)

- 一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。
- 接下来的其他操作或故障不应该对其执行结果有任何影响。

事务的特性

❑ 保证事务**ACID**特性是事务处理的任务

❑ 破坏事务**ACID**特性的因素

➤ 多个事务并行运行时，不同事务的操作交叉执行

- 数据库管理系统必须保证多个事务的交叉运行不影响这些事务的隔离性

➤ 事务在运行过程中被强行停止

- 数据库管理系统必须保证被强行终止的事务对数据库和其他事务没有任何影响

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

故障和数据库恢复

❑ 故障是不可避免的

- 计算机硬件故障
- 软件的错误
- 操作员的失误
- 恶意的破坏

❑ 故障的影响

- 运行事务非正常中断，影响数据库中数据的正确性
- 破坏数据库，全部或部分丢失数据

❑ 所谓的‘运行事务’是指已经开始、但还没有被提交或回滚、正在运行中的事务，也被称为‘**活跃事务**’(active transaction)。那些已经运行结束的事务被称为‘非活跃事务’(inactive transaction)。

❑ 数据库的恢复

- 数据库管理系统必须具有把数据库从错误状态恢复到某一已知的正确状态(亦称为一致状态或完整状态)的功能，这就是数据库的恢复管理系统对故障的对策

❑ 恢复子系统是数据库管理系统的一个重要组成部分

❑ 恢复技术是衡量系统优劣的重要指标

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

❑ 故障的种类

- 事务内部的故障
- 系统故障
- 介质故障
- 计算机病毒

❑ 各类故障，对数据库的影响有两种可能性

- 一是数据库本身被破坏
- 二是数据库没有被破坏，但数据可能不正确，这是由于事务的运行被非正常终止造成的。

事务内部的故障 1

❑ 例如，银行转账事务，这个事务把一笔金额从一个账户甲转给另一个账户乙。

```
BEGIN TRANSACTION
```

```
读账户甲的余额BALANCE;
```

```
BALANCE = BALANCE - AMOUNT; /*AMOUNT 为转账金额*/
```

```
IF ( BALANCE < 0 ) THEN
```

```
{
```

```
    打印‘金额不足，不能转账’; /*事务内部可能造成事务被回滚的情况*/
```

```
    ROLLBACK; /*撤销刚才的修改，恢复事务*/
```

```
}
```

```
ELSE
```

```
{
```

```
    读账户乙的余额BALANCE1;
```

```
BALANCE1=BALANCE1+AMOUNT;
```

```
写回BALANCE1;
```

```
COMMIT; /*提交事务的修改结果*/
```

```
}
```

事务内部的故障 2

- ❑ 前一页的‘银行转账’事务，所包括的两个更新操作要么全部完成要么全部不做。否则就会使数据库处于不一致状态，例如只把账户甲的余额减少了而没有把账户乙的余额增加。
- ❑ 在这段程序中若产生账户甲余额不足的情况，应用程序可以发现并让事务滚回，撤销已作的修改，恢复数据库到正确状态。
- ❑ 事务内部更多的故障是非预期的，是不能由应用程序处理的。
 - 运算溢出
 - 并发事务发生死锁而被选中撤销该事务
 - 违反了某些完整性限制而被终止等
- ❑ 事务故障仅指这类非预期的故障

事务故障的恢复

❑ 事务故障意味着

- 事务没有达到预期的终点(**COMMIT**或者显式的**ROLLBACK**)
- 数据库可能处于不正确状态。

❑ 事务故障的恢复：事务撤消（**UNDO**）

- 强行回滚（**ROLLBACK**）该事务
- 撤销该事务已经作出的任何对数据库的修改，使得该事务象根本没有启动一样

系统故障

❑ 系统故障，称为软故障，是指造成系统停止运转的任何事件（特定类型的硬件错误（如CPU故障）、操作系统故障、数据库管理系统代码错误、系统断电），使得系统要重新启动。

- 整个系统的正常运行突然被破坏
- 所有正在运行的事务都非正常终止
- 不破坏数据库
- 内存中数据库缓冲区的信息全部丢失

系统故障的恢复：UNDO & REDO

- ❑ 发生系统故障时，一些尚未完成的事务的结果可能已送入物理数据库，造成数据库可能处于不正确状态。
 - 恢复策略：系统重新启动时，恢复程序让所有非正常终止的事务回滚，强行撤消（**UNDO**）所有未完成事务

- ❑ 发生系统故障时，有些已完成的事务可能有一部分甚至全部留在缓冲区，尚未写回到磁盘上的物理数据库中，系统故障使得这些事务对数据库的修改部分或全部丢失
 - 恢复策略：系统重新启动时，恢复程序需要重做（**REDO**）所有已提交的事务

介质故障

- ❑ 介质故障，称为硬故障，指外存故障
 - 磁盘损坏
 - 磁头碰撞
 - 瞬时强磁场干扰
- ❑ 介质故障破坏数据库或部分数据库，并影响正在存取这部分数据的所有事务
- ❑ 介质故障比前两类故障的可能性小得多，但破坏性大得多

计算机病毒

❑ 计算机病毒

- 一种人为的故障或破坏，是一些恶作剧者研制的一种计算机程序
- 可以繁殖和传播，造成对计算机系统包括数据库的危害

❑ 计算机病毒已成为计算机系统的主要威胁，自然也是数据库系统的主要威胁

❑ 数据库一旦被破坏仍要用恢复技术把数据库加以恢复

故障小结

- ❑ 各类故障，对数据库的影响有两种可能性
 - 一是数据库本身被破坏
 - 二是数据库没有被破坏，但数据可能不正确，这是由于事务的运行被非正常终止造成的。

恢复

❑ 恢复操作的基本原理：冗余

- 利用存储在系统别处的冗余数据来重建数据库中已被破坏或不正确的那部分数据

❑ 恢复的实现技术：复杂

- 一个大型数据库产品，恢复子系统的代码要占全部代码的10%以上

❑ 恢复机制涉及的关键问题

- 如何建立冗余数据（数据转储，登记日志文件）
- 如何利用这些冗余数据实施数据库恢复

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

□ 恢复机制涉及的关键问题

1. 如何建立冗余数据

- 数据转储 (backup)
- 登记日志文件 (logging)

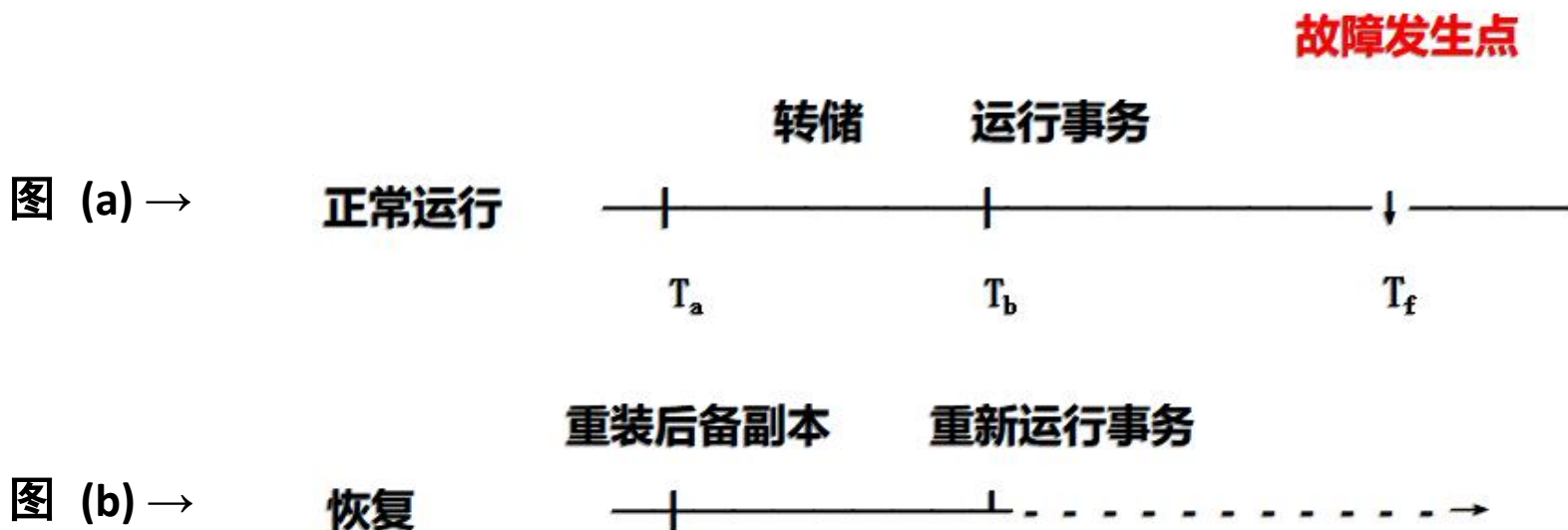
2. 如何利用这些冗余数据实施数据库恢复

数据转储 1

- ❑ ‘转储’是指数据库管理员定期地将整个数据库复制到磁带、磁盘或其他存储介质上保存起来的过程。
- ❑ 执行数据转储所生成的备用数据文本称为‘后备副本’(backup)或‘后援副本’
- ❑ 数据库遭到破坏后可以将后备副本重新装入
- ❑ 重装后备副本只能将数据库恢复到转储时的状态
- ❑ 要想恢复到故障发生时的状态，必须重新运行自转储以后的所有更新事务

数据转储 2 (转储和恢复)

- 在下图(a)中:
- 系统在 T_a 时刻停止运行事务, 进行数据库转储
 - 在 T_b 时刻转储完毕, 得到 T_b 时刻的数据库一致性副本
 - 系统运行到 T_f 时刻发生故障



- 在上图(b)中:
- 为恢复数据库, 首先由数据库管理员重装数据库后备副本, 将数据库恢复至 T_b 时刻的状态
 - 重新运行自 $T_b \sim T_f$ 时刻的所有更新事务, 把数据库恢复到故障发生前的一致状态

- ❑ 静态转储 与 动态转储
- ❑ 海量转储 与 增量转储

□ 静态转储

- 在系统中无运行事务时进行的转储操作
- 转储开始时数据库处于一致性状态
- 转储期间不允许对数据库的任何存取、修改活动
- 得到的一定是一个数据一致性的副本

□ 优点：实现简单

□ 缺点：降低了数据库的可用性

- 转储必须等待正运行的用户事务结束
- 新的事务必须等转储结束

转储方法 - 动态转储

□ 动态转储

- 转储操作与用户事务并发进行
- 转储期间允许对数据库进行存取或修改

□ 动态转储的优点

- 不用等待正在运行的用户事务结束
- 不会影响新事务的运行

□ 动态转储的缺点

- 不能保证副本中的数据正确有效
- 例如：

- 在转储期间的某时刻 T_c ，系统把数据 $A=100$ 转储到磁带上，而在下一时刻 T_d ，某一事务将 A 改为200。
- 后备副本上的 A 过时了

□ 利用动态转储得到的副本进行故障恢复

- 需要把动态转储期间各事务对数据库的修改活动登记下来，建立日志文件
- 后备副本加上日志文件就能把数据库恢复到某一时刻的正确状态

转储方法 - 海量转储 与 增量转储

- ❑ 海量转储: 每次转储全部数据库
- ❑ 增量转储: 只转储上次转储后更新过的数据
- ❑ 海量转储与增量转储比较
 - 从恢复角度看, 使用海量转储得到的后备副本进行恢复往往更方便
 - 如果数据库很大, 事务处理又十分频繁, 则增量转储方式更实用更有效

日志文件

❑ 日志文件(log file)是用来记录事务对数据库的更新操作的文件

❑ 日志文件的格式

- 以记录为单位的日志文件
- 以数据块为单位的日志文件

❑ 用途

- 进行事务故障恢复
- 进行系统故障恢复
- 协助后备副本进行介质故障恢复

日志文件的内容

- ❑ 在数据库的日志文件中，每一条日志记录(log record)中需要含有日志记录类型标识和事务标识（用来标识该日志记录属于哪一个事务）
- ❑ 主要有以下几种类型的日志记录
 - 事务开始标记：事务的开始日志（BEGIN TRANSACTION）
 - 事务结束标记：事务的提交日志（COMMIT TRANSACTION）或放弃日志（ROLLBACK TRANSACTION）
 - 事务更新日志：事务对数据库的更新操作的执行记录（也被称为事务的‘更新日志’）
 - 其他日志记录：如检查点日志(CHECKPOINT)、保存点日志(SAVEPOINT)等。
- ❑ 根据在事务更新日志中记载的内容，日志文件的格式被划分为“以记录为单位”和“以数据块为单位”的两种类型

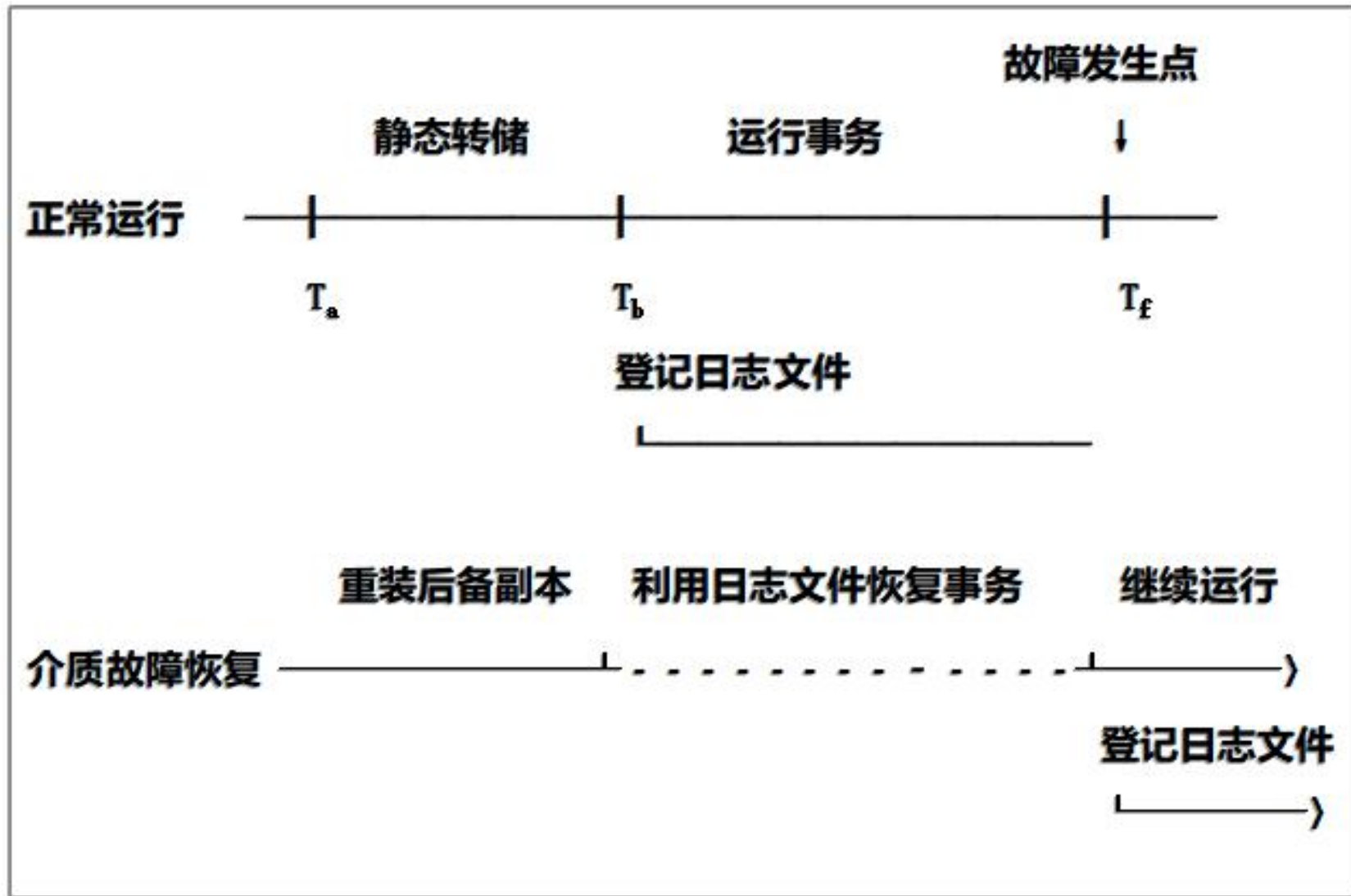
更新日志记录的内容

- ❑ 在以记录为单位的日志文件中，每条更新日志记录的内容
 - 事务标识（标明是哪个事务）
 - 操作类型（插入、删除或修改）
 - 操作对象（记录内部标识）
 - 前像值(before image): 更新前数据的旧值（对插入操作而言，此项为空值）
 - 后像值(after image): 更新后数据的新值（对删除操作而言，此项为空值）
- ❑ 在以数据块为单位的日志文件中，每条更新日志记录的内容
 - 事务标识
 - 被更新的数据块（包括 前像值&后像值）

□ 具体作用

- 事务故障恢复和系统故障恢复必须用日志文件。
- 在动态转储方式中必须建立日志文件，后备副本和日志文件结合起来才能有效地恢复数据库。
- 在静态转储方式中，也可以建立日志文件。
 - 当数据库毁坏后可重新装入后备副本把数据库恢复到转储结束时刻的正确状态
 - 利用日志文件，把已完成的事务进行重做处理（REDO）
 - 对故障发生时尚未完成的事务进行撤销处理（UNDO）
 - 不必重新运行那些已完成的事务程序就可把数据库恢复到故障前某一时刻的正确状态

日志文件的作用 2



利用日志文件恢复

登记日志文件 1

□ 为保证数据库是可恢复的，登记日志文件时必须遵循两条原则

1. 登记的次序严格按**并发事务执行的时间次序**
2. 必须**先写日志文件，后写数据库**
 - 写日志文件操作：把“表示某个数据库修改操作的日志记录”写到日志文件中
 - 写数据库操作：把“对数据的修改结果”写到数据库中

➤ 说明：

- “写日志文件”和“写数据库”是两个不同但又相互有关联的操作
- 每一个“写数据库”操作都有对应的一个“写日志文件”操作，他们两者之间的执行顺序决定了数据库是否具有可恢复性

□ 为什么要先写日志文件？

- 写数据库和写日志文件是两个不同的操作
- 在这两个操作之间可能发生故障
- 如果先写了数据库修改，而在日志文件中没有登记下这个修改，则以后就无法恢复（**UNDO**）这个修改了
- 如果先写日志，但没有修改数据库，按日志文件恢复时只不过是多执行一次不必要的**UNDO**操作，并不会影响数据库的正确性

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

- ❑ 10.5.1 事务故障的恢复
- ❑ 10.5.2 系统故障的恢复
- ❑ 10.5.3 介质故障的恢复

10.5.1 事务故障的恢复 (UNDO)

- ❑ 事务故障：事务在运行至正常终止点前被终止
 - 在数据库管理系统正常运行过程中，放弃某个事务（以 **ROLLBACK** 方式结束一个事务）
- ❑ 恢复方法
 - 由恢复子系统利用日志文件撤消（**UNDO**）此事务已对数据库进行的修改
- ❑ 事务故障的恢复由系统自动完成，对用户是透明的，不需要用户干预

10.5.1 事务故障的恢复 (UNDO) 步骤

- (1) 反向扫描文件日志（即从最后向前扫描日志文件），查找该事务的更新操作。
- (2) 对该事务的更新操作执行逆操作。即将日志记录中“更新前的值”写入数据库。
 - 插入操作，“更新前的值”为空，则相当于做删除操作
 - 删除操作，“更新后的值”为空，则相当于做插入操作
 - 若是修改操作，则相当于用修改前值代替修改后值
- (3) 继续反向扫描日志文件，查找该事务的其他更新操作，并按照步骤2中的方法做同样处理。
- (4) 如此处理下去，直至读到此事务的开始标记，然后回到日志文件的尾部，为该事务写入一条事务放弃的结束标记，事务故障恢复就完成了。

10.5.2 系统故障的恢复

❑ 系统故障造成数据库不一致状态的原因

- 未完成事务对数据库的更新可能已写入数据库
- 已提交事务对数据库的更新可能还留在缓冲区没来得及写入数据库

❑ 恢复方法

- **Undo** 故障发生时未完成的事务
- **Redo** 已完成的事务

❑ 系统故障的恢复由系统在重新启动时自动完成，不需要用户干预

10.5.2 系统故障的恢复步骤

- (1) 正向扫描日志文件（即从头开始扫描日志文件），收集得到以下两个事务队列
 - 重做(REDO) 队列: 在故障发生前已经提交的事务
 - 这些事务既有BEGIN TRANSACTION记录，也有COMMIT记录
 - 撤销 (UNDO)队列: 故障发生时尚未完成的事务
 - 这些事务只有BEGIN TRANSACTION记录，无相应的COMMIT记录
- (2) 对撤销(UNDO)队列事务进行撤销(UNDO)处理
 - 反向扫描日志文件，对每个撤销事务的更新操作执行逆操作
 - 即将日志记录中“更新前的值”写入数据库
- (3) 对重做(REDO)队列事务进行重做(REDO)处理
 - 正向扫描日志文件，对每个重做事务的更新日志，重新执行登记的操作
 - 即将日志记录中“更新后的值”写入数据库
- (4) 对撤销(UNDO)队列中的每一个没有结束标记的撤销事务，添加一条事务放弃日志记录(ROLLBACK TRANSACTION)，系统故障的恢复处理结束。

10.5.3 介质故障的恢复

- ❑ 介质故障的恢复的工作
 - 重装数据库
 - 重做已完成的事务

- ❑ 介质故障的恢复需要数据库管理员介入

- ❑ 数据库管理员的工作
 - 重装最近转储的数据库副本和有关的各日志文件副本
 - 执行系统提供的恢复命令

- ❑ 具体的恢复操作仍由数据库管理系统完成

10.5.3 介质故障的恢复步骤

- (1) 装入最新的后备数据库副本(离故障发生时刻最近的转储副本)，使数据库恢复到最近一次转储时的一致性状态。
 - 对于静态转储的数据库副本，装入后数据库即处于一致性状态
 - 对于动态转储的数据库副本，还须同时装入转储时刻的日志文件副本，利用恢复系统故障的方法（即REDO+UNDO），才能将数据库恢复到一致性状态。
- (2) 装入有关的日志文件副本(转储结束时刻的日志文件副本)，重做已完成的事务。
 - 首先扫描日志文件，找出故障发生时已提交的事务的标识，将其记入重做队列。
 - 然后正向扫描日志文件，对重做队列中的所有事务进行重做处理。即将日志记录中“更新后的值”写入数据库。

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

❑ 恢复的两个问题

- 搜索整个日志将耗费大量的时间
- 重做处理：重新执行，浪费了大量时间

❑ 具有检查点（checkpoint）的恢复技术

- 在日志文件中增加检查点记录（checkpoint）
- 增加重新开始文件
- 恢复子系统在登录日志文件期间动态地维护日志

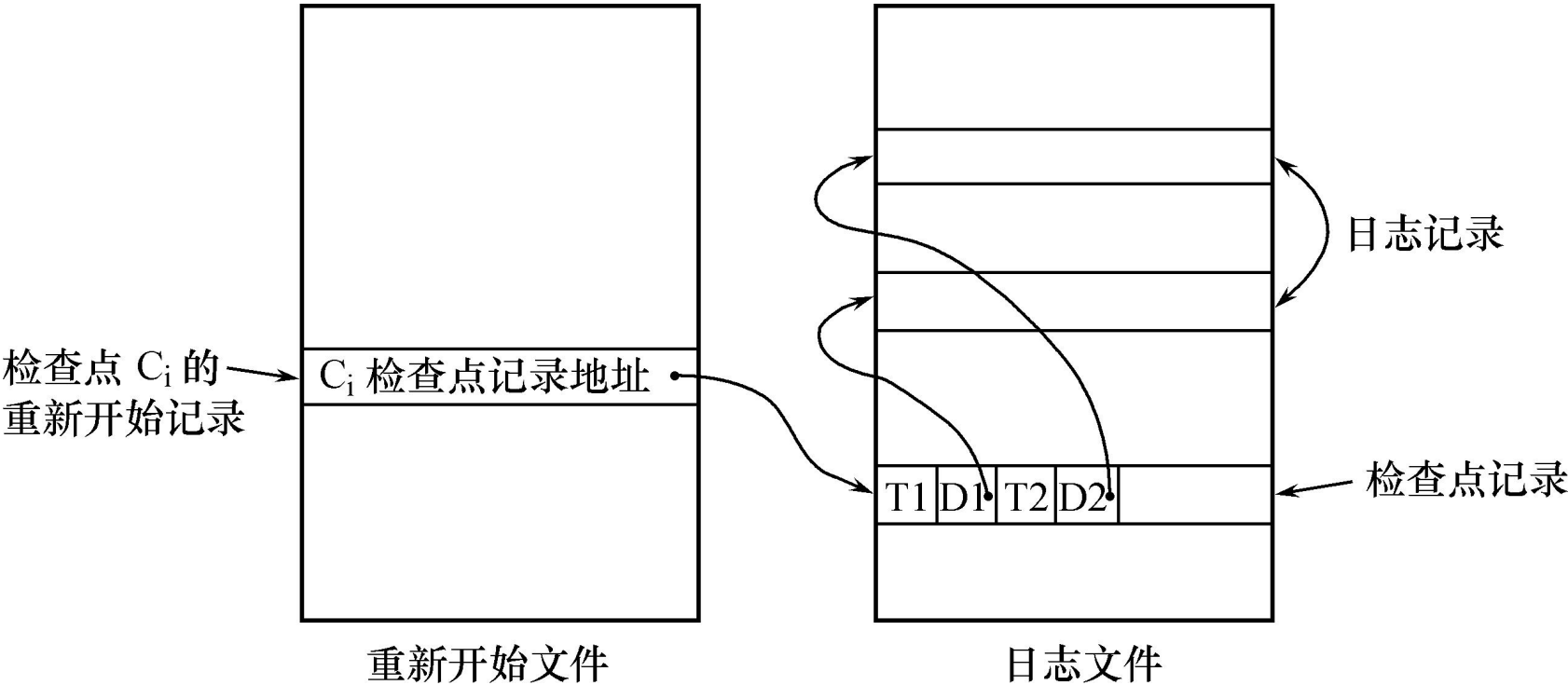
❑ 检查点记录的内容

- 建立检查点时刻所有正在执行的事务清单
- 这些事务最近一个日志记录的地址

❑ 重新开始文件的内容

- 记录各个检查点记录在日志文件中的地址

检查点技术 2



具有检查点的日志文件和重新开始文件

动态维护日志文件的方法

❑ 动态维护日志文件的方法

➤ 周期性地执行如下操作：建立检查点，保存数据库状态。

➤ 具体步骤是：

- ① 将当前日志缓冲区中的所有日志记录写入磁盘的日志文件上
- ② 在日志文件中写入一个检查点记录
- ③ 将当前数据缓冲区的所有数据记录写入磁盘的数据库中
- ④ 把检查点记录在日志文件中的地址写入一个重新开始文件

建立检查点

❑ 恢复子系统可以定期或不定期地建立检查点,保存数据库状态

➤ 定期

- 按照预定的一个时间间隔,如每隔一小时建立一个检查点

➤ 不定期

- 按照某种规则,如日志文件已写满一半建立一个检查点

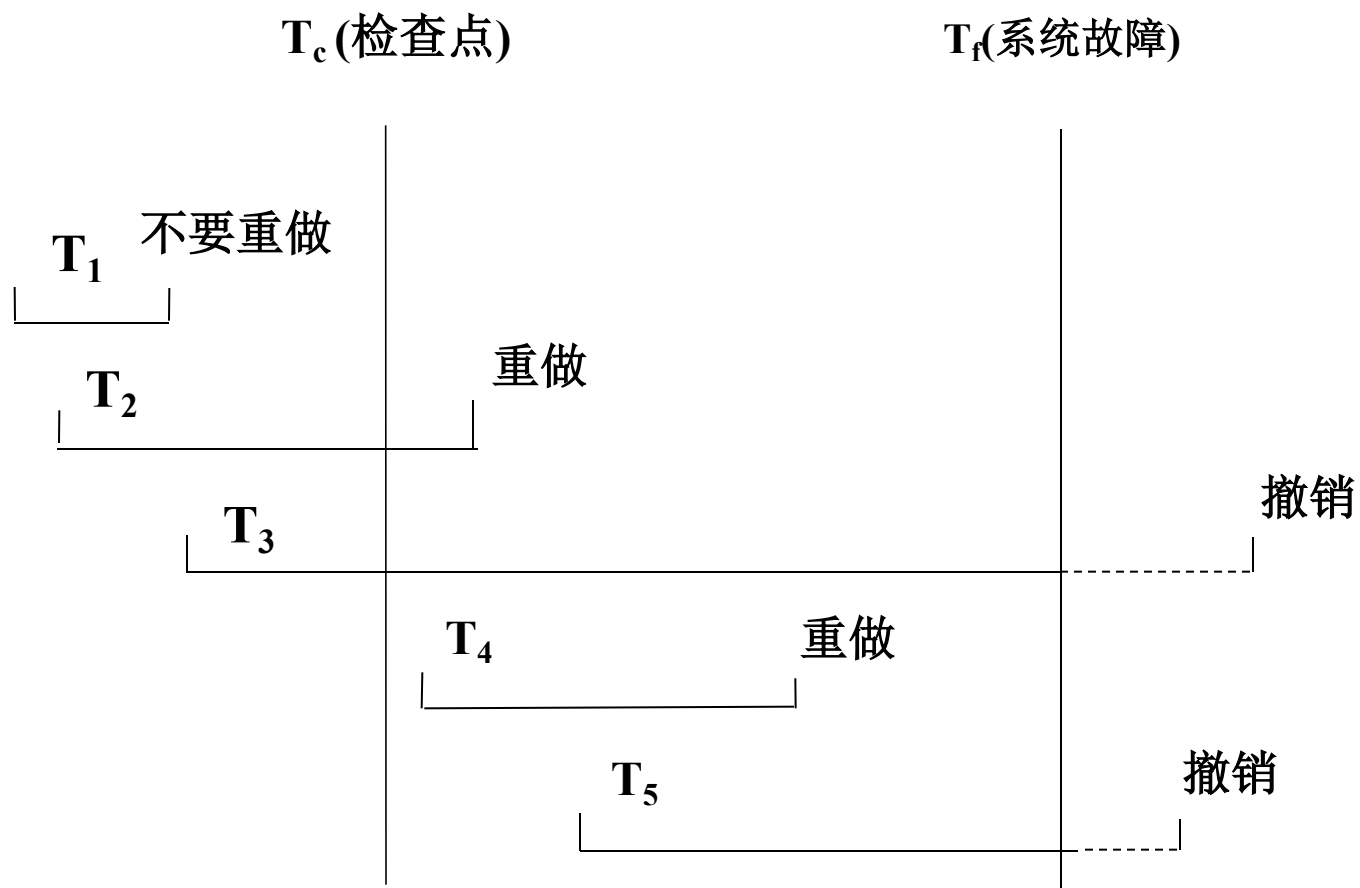
利用检查点的恢复策略 1

❑ 使用检查点方法可以改善恢复效率

- 当事务T在一个检查点之前提交，T对数据库所做的修改已写入数据库
- 写入时间是在这个检查点建立之前或在这个检查点建立之时
- 在进行恢复处理时，没有必要对事务T执行重做操作

利用检查点的恢复策略 2

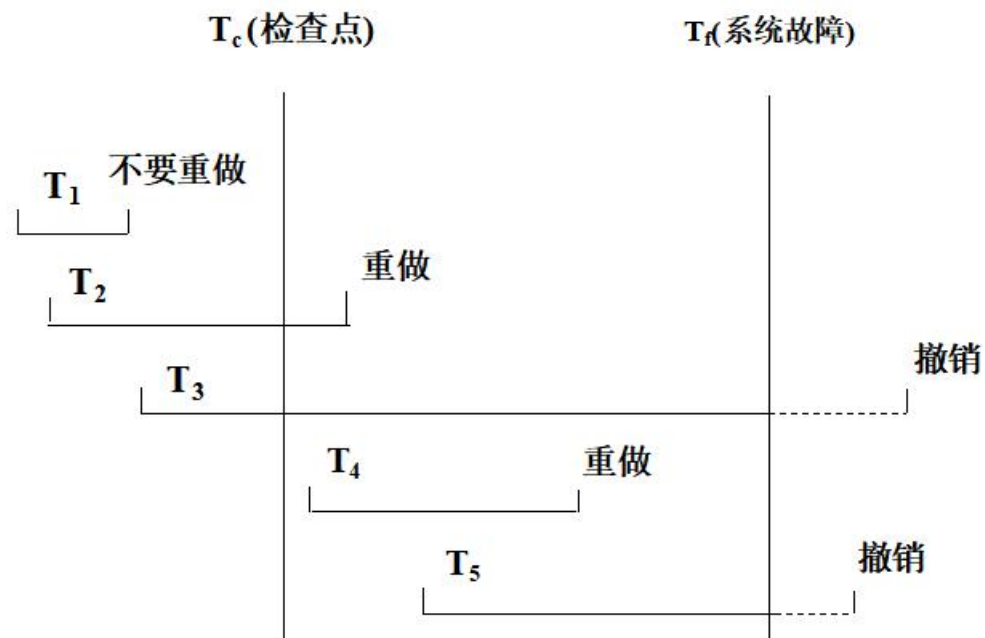
系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略



利用检查点的恢复策略 3

故障状态

- T_1 : 在检查点之前提交
- T_2 : 在检查点之前开始执行, 在检查点之后、故障点之前提交
- T_3 : 在检查点之前开始执行, 在故障点时还未完成
- T_4 : 在检查点之后开始执行, 在故障点之前提交
- T_5 : 在检查点之后开始执行, 在故障点时还未完成

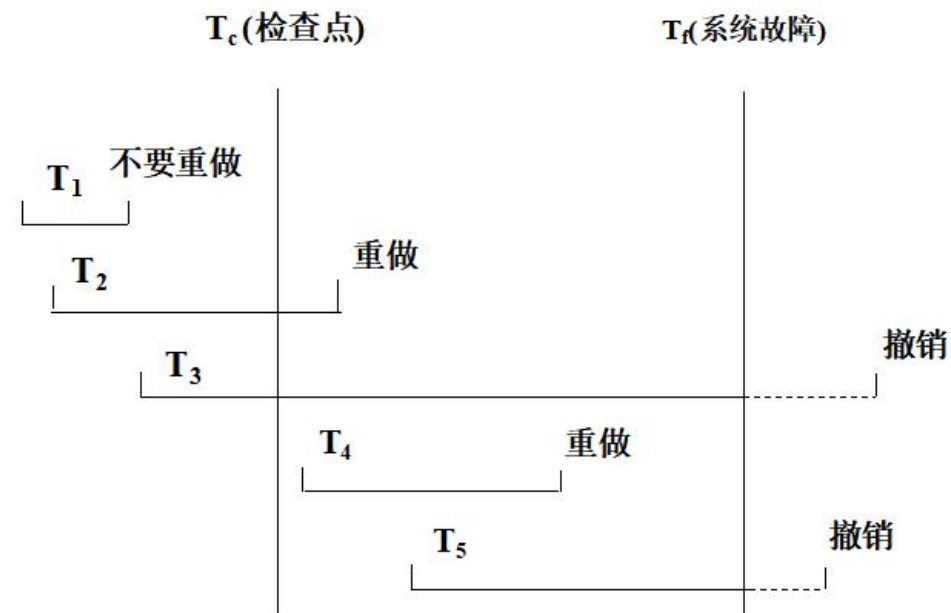


恢复策略

- T_3 和 T_5 在故障发生时还未完成, 所以予以撤销
- T_2 和 T_4 在检查点之后才提交, 它们对数据库所做的修改在故障发生时可能还在缓冲区中, 尚未写入数据库, 所以要重做
- T_1 在检查点之前已提交, 所以不必执行重做操作

利用检查点的恢复步骤

- ❑ 从重新开始文件中找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录
- ❑ 由该检查点记录得到检查点建立时刻所有正在执行的事务清单ACTIVE-LIST
 - 建立两个事务队列：UNDO-LIST 和 REDO-LIST
 - 把ACTIVE-LIST暂时放入UNDO-LIST队列，REDO队列暂为空。



- (1) 从检查点开始正向扫描日志文件，直到日志文件结束
 - 如有新开始的事务 T_i ，把 T_i 暂时放入UNDO-LIST队列
 - 如有提交的事务 T_j ，把 T_j 从UNDO-LIST队列移到REDO-LIST队列
- (2) 对UNDO-LIST中的每个事务执行UNDO操作（逆向扫描日志文件，直至UNDO-LIST队列中所有事务的起点）
- (3) 对REDO-LIST中的每个事务执行REDO操作（从REDO-LIST中最先开始的那个事务的起点，正向扫描日志文件）
- (4) 在日志文件的尾部，为UNDO-LIST中的每个事务补写一条放弃日志(ROLLBACK)。

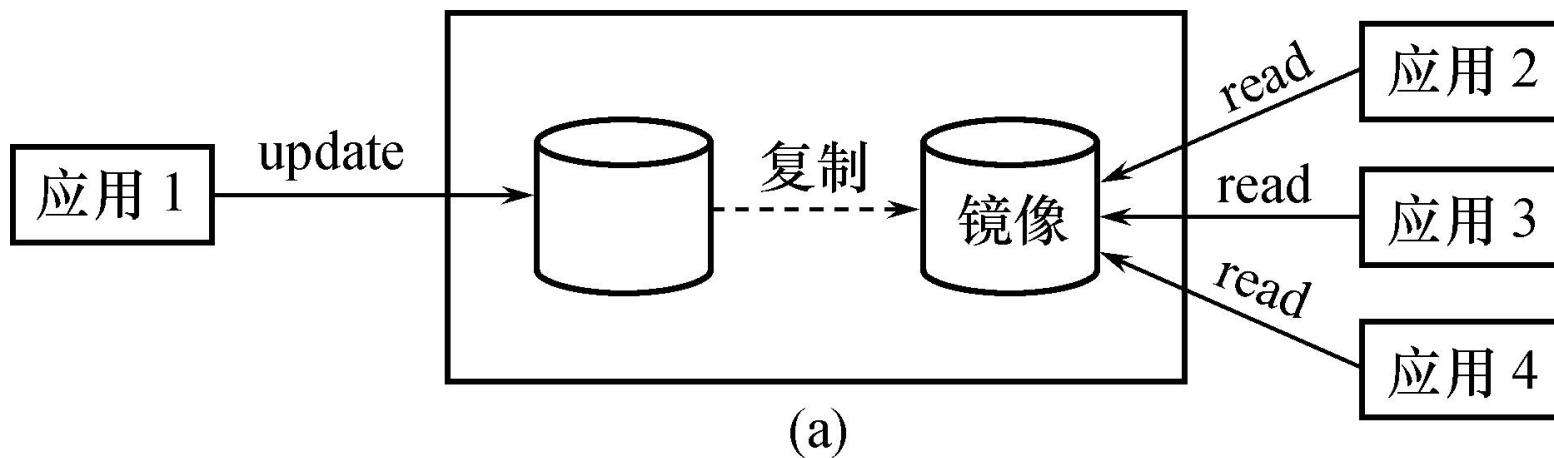
- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

- ❑ 介质故障是对系统影响最为严重的一种故障，严重影响数据库的可用性
 - 介质故障恢复比较费时
 - 为预防介质故障，数据库管理员必须周期性地转储数据库

- ❑ 提高数据库可用性的解决方案
 - 数据库镜像 (Mirror)

□ 数据库镜像

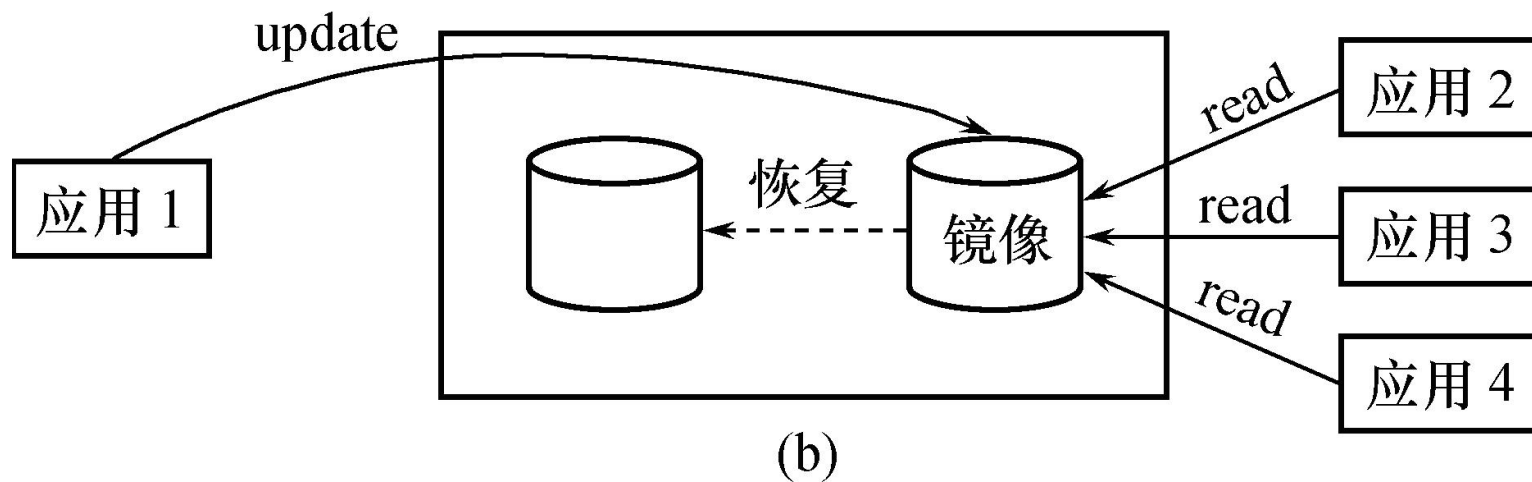
- 数据库管理系统自动把整个数据库或其中的关键数据复制到另一个磁盘上
- 数据库管理系统自动保证镜像数据与主数据的一致性
- 每当主数据库更新时，数据库管理系统自动把更新后的数据复制过去



数据库镜像的用途 1

❑ 出现介质故障时

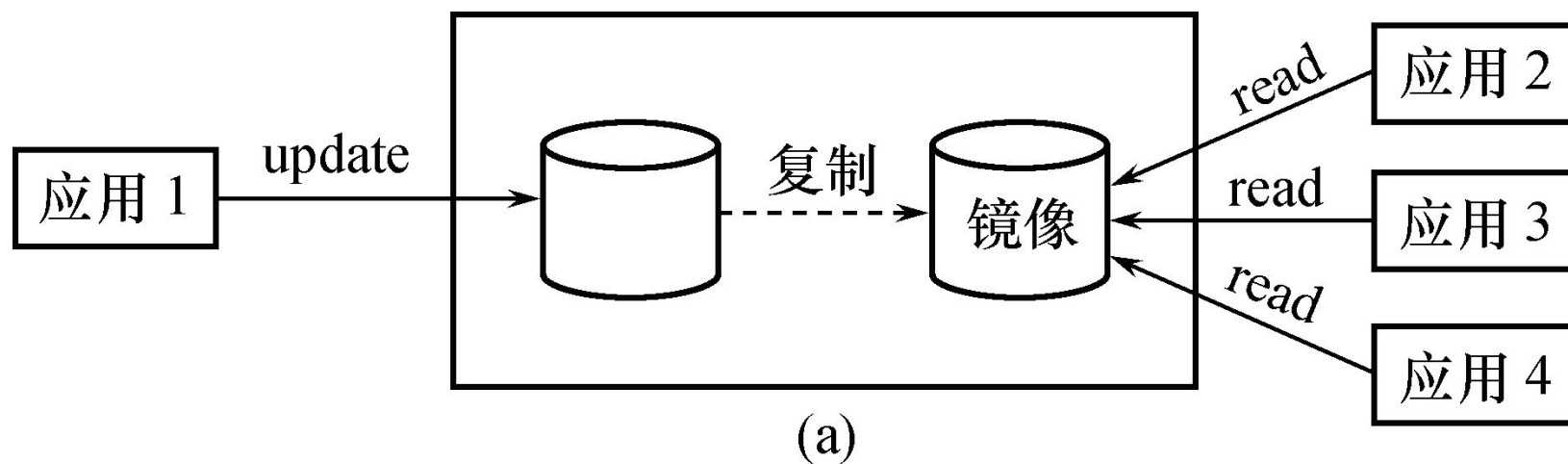
- 可由镜像磁盘继续提供使用
- 同时数据库管理系统自动利用镜像磁盘数据进行数据库的恢复
- 不需要关闭系统和重装数据库副本



数据库镜像的用途 2

□ 没有出现故障时

- 可用于并发操作（提高系统的并发度）
- 一个用户对数据加排他锁修改数据时，其他用户可以读镜像数据库上的数据，而不必等待该用户释放锁（需要新的并发控制协议）



- ❑ 频繁地复制数据自然会降低系统运行效率
 - 在实际应用中用户往往只选择对关键数据和日志文件镜像
 - 不是对整个数据库进行镜像

- ❑ 10.1 事务的基本概念
- ❑ 10.2 数据库恢复概述
- ❑ 10.3 故障的种类
- ❑ 10.4 恢复的实现技术
- ❑ 10.5 恢复策略
- ❑ 10.6 具有检查点的恢复技术
- ❑ 10.7 数据库镜像
- ❑ 10.8 小结

10.8 小结

❑ 事务的概念和性质

- 事务是数据库的逻辑工作单位
- 数据库管理系统保证系统中一切事务的原子性、一致性、隔离性和持续性，就保证了事务处于一致状态

❑ 故障的种类：事务故障、系统故障、介质故障

❑ 恢复中最经常使用的技术：数据库转储、登记日志文件、数据库镜像

❑ 恢复的基本原理

- 利用存储在后备副本、日志文件和数据库镜像中的冗余数据来重建数据库

❑ 事务

- 不仅是恢复的基本单位
- 也是并发控制的基本单位